Software Testing & CI/CD

CMP9134: Software Engineering

Dr Francesco Del Duchetto
Lecturer in Robotics and Autonomous Systems

University of Lincoln

27 March 2026

Today's Agenda

1 Recap & Introduction

2 The Philosophy of Testing

3 Testing Levels & The V-Model

4 Test Design Techniques

5 Writing Tests & Mocking

6 Testing Types

7 Test Automation & CI/CD

8 Next Steps

Agenda

1 Recap & Introduction
2 The Philosophy of Testing
3 Testing Levels & The V-Model
4 Test Design Techniques
5 Writing Tests & Mocking
6 Testing Types
7 Test Automation & CI/CD
8 Next Steps

Recap: Containerisation & Docker

Last week, Prof. Marc Hanheide introduced us to the reality of deploying software in the real world using **Containerisation**.

- **The Problem:** ”It works on my machine!” Discrepancies between developer laptops and production servers lead to catastrophic deployment failures.
- **The Solution:** Docker containers package the application code along with all its dependencies, libraries, and OS configurations into a single, immutable, reproducible image.
- **Microservices:** Using `docker-compose` to orchestrate multiple independent services (e.g., your Frontend, your FastAPI Backend, and the Virtual Robot Simulator).

Bridging Containers and Testing

Why did we learn about Docker before Software Testing?

Because **Automated Testing** relies entirely on predictable environments!

- If a test passes on your laptop but fails on the server, the test is useless.
- By running our automated tests *inside* Docker containers, we guarantee that the test environment is 100% identical to the production environment.

Today, we look at the theory, design, and automation of Software Testing.

Today's Learning Objectives

By the end of this comprehensive lecture, you should be able to:

1. **Explain** the philosophy of testing and the fundamental difference between Verification and Validation.
2. **Map** testing activities to the Software Development Life Cycle (SDLC) using the V-Model.
3. **Apply** rigorous Test Design Techniques (Black-box Boundary Value Analysis vs. White-box Coverage).
4. **Implement** Unit Tests using the Arrange-Act-Assert pattern and Mocking.
5. **Architect** a Continuous Integration / Continuous Deployment (CI/CD) pipeline.

Agenda

1 Recap & Introduction

2 The Philosophy of Testing

3 Testing Levels & The V-Model

4 Test Design Techniques

5 Writing Tests & Mocking

6 Testing Types

7 Test Automation & CI/CD

8 Next Steps

What is Software Testing?

Testing is the process of evaluating a system or its component(s) with the intent to find whether it satisfies the specified requirements or not.

Dijkstra's Theorem

"Program testing can be used to show the presence of bugs, but never to show their absence!" — Edsger W. Dijkstra (1970)

The Tester's Mindset:

- A developer writes code to make the system **work**.
- A tester writes code to make the system **break**.
- A successful test is one that uncovers an as-yet-undiscovered error. An unsuccessful test is one that finds no errors.

Real-World Failures

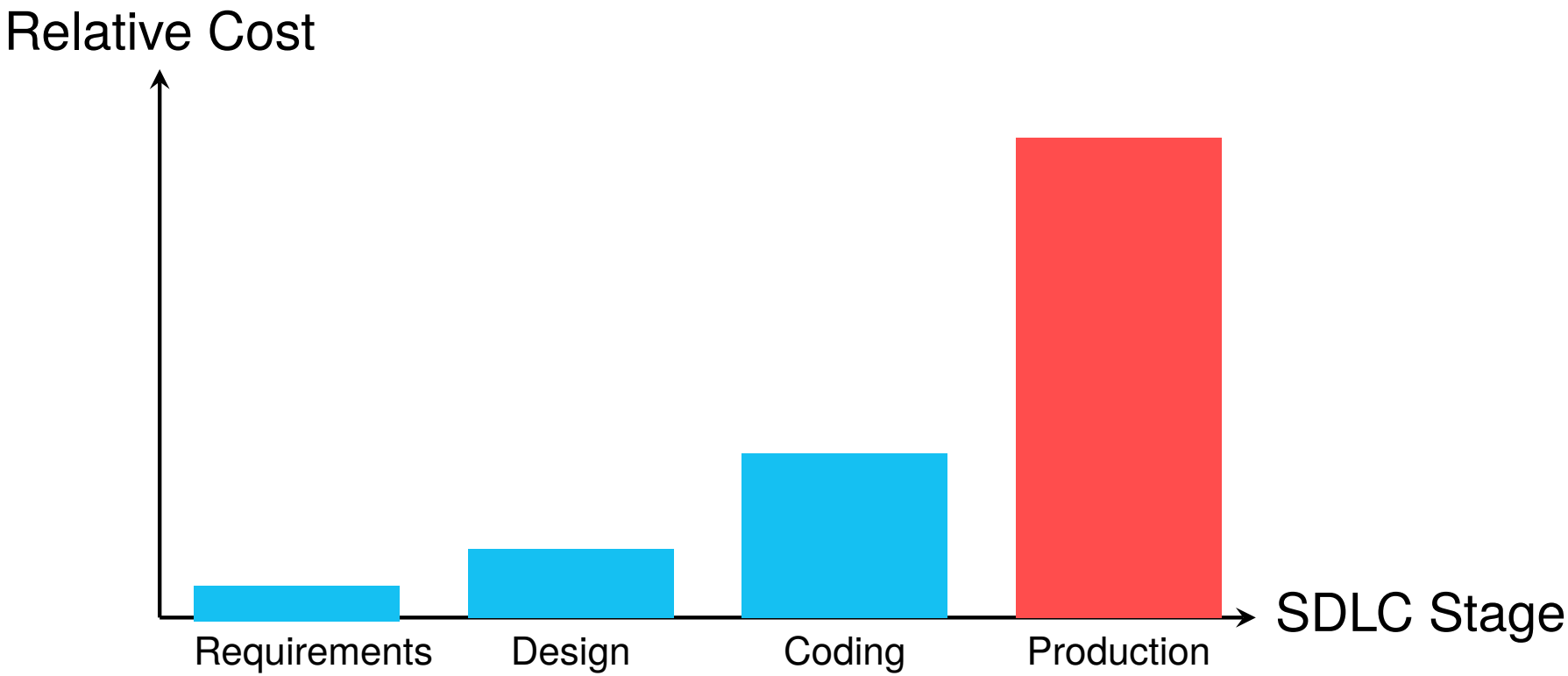
Software bugs are inevitable, but shipping them to production is a choice.

Why Testing is Critical:

- **Public Embarrassment:** AI demos failing live on stage (e.g., Google's Gemini launch anomalies, LG's robot refusing to respond at CES).
- **Financial Ruin:** The Knight Capital Group lost $440 million in 45 minutes due to an untested, obsolete trading algorithm being deployed.
- **Safety Risks:** The Ariane 5 rocket explosion ($370 million lost). A 64-bit floating-point number was converted to a 16-bit integer, causing an overflow. A simple unit test would have caught this!

The Cost of Fixing Defects

Why do we emphasize testing early and often? The later a bug is found in the SDLC, the more exponentially expensive it becomes to fix.



Fixing an architectural logic error during the design phase costs $10 in time. Fixing that same error after the software is deployed to thousands of users costs $10,000+ in

Verification vs. Validation

These two terms are often used interchangeably, but in Software Engineering, they mean very different things:

Verification: ”Are we building the product right?”

- Checks if the software conforms to its technical specifications and architecture.
- **Focus:** Code reviews, Unit tests, Integration tests.

Validation: ”Are we building the right product?”

- Checks if the software actually fulfills the client’s business needs and solves the real-world problem.
- **Focus:** User Acceptance Testing (UAT), Usability testing.

Agenda

1 Recap & Introduction

2 The Philosophy of Testing

3 Testing Levels & The V-Model

4 Test Design Techniques

5 Writing Tests & Mocking

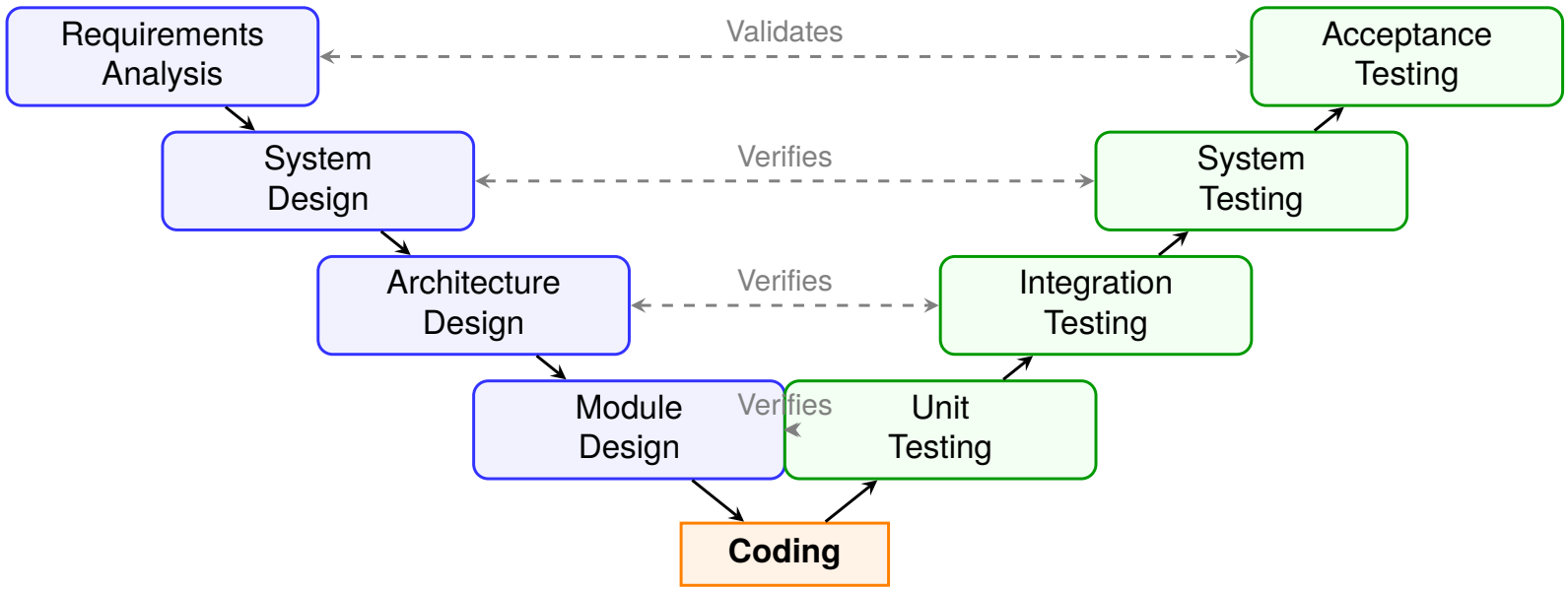
6 Testing Types

7 Test Automation & CI/CD

8 Next Steps

The V-Model

The V-Model demonstrates how every stage of software design has a corresponding, parallel stage of software testing. You do not wait until the end to write tests!



Testing Level 1: Unit Testing

Definition: Tests individual components, methods, or functions in total isolation from the rest of the system.

- **Who does it?** The developers themselves, usually while writing the code (or before, if using Test-Driven Development - TDD).
- **Characteristics:** They must be extremely fast (executing in milliseconds), deterministic, and have no external dependencies (no real databases, no real network calls).
- **Example:** Testing a Python function that calculates the distance between two coordinates.

Testing Level 2: Integration Testing

Definition: Tests how different modules, services, or containers work when connected to one another.

- **The Problem it solves:** Unit tests prove the modules work alone, but they might fail when passing data to each other.
- **Who does it?** Developers or dedicated Quality Assurance (QA) Automation engineers.
- **Example:** Does the Python backend correctly establish a connection to the PostgreSQL database container? Can the backend properly parse the JSON response from the external Virtual Robot API?

Testing Levels 3 & 4: System and Acceptance

System Testing

Evaluates the complete, fully integrated software product against the system requirements.

- This is End-to-End (E2E) testing.
- *Example:* Using an automated browser tool (like Selenium or Cypress) to click the ”Login” button, navigate to the dashboard, and verify the robot status appears on the screen.

Acceptance Testing

Conducted by the client to verify the system satisfies business needs (Validation).

- **Alpha Testing:** Performed by internal staff in a controlled environment.
- **Beta Testing:** Released to a limited set of external end-users in a real-world, uncontrolled environment to catch edge-case bugs before full launch.

Agenda

1 Recap & Introduction

2 The Philosophy of Testing

3 Testing Levels & The V-Model

4 Test Design Techniques

5 Writing Tests & Mocking

6 Testing Types

7 Test Automation & CI/CD

8 Next Steps

How do we design tests?

Writing tests randomly (”Happy Path” testing) is inefficient and leaves massive vulnerabilities. We use structured techniques to ensure maximum coverage with the minimum number of test cases.

Black-Box Testing

- The tester knows **nothing** about the internal code.
- Focuses purely on Inputs and Outputs.
- Validates functional requirements.
- E.g., User provides a username/password, expects a success message.

White-Box Testing

- The tester has **full access** to the source code.
- Focuses on internal logic, loops, variables, and conditional branches.
- Verifies structural integrity.
- E.g., Ensuring an `if/else` statement is fully tested.

Black-Box: Equivalence Partitioning

The Problem: You cannot test every possible input. If an API accepts an integer from 1 to 10,000, writing 10,000 tests takes too much time and processing power.
The Solution: Divide inputs into ”Equivalence Classes” where the program is expected to behave the exact same way for every value in that class. You only need to test *one* value from each class!

Example: A robot accepts a speed command from 0 to 100 km/h.

- **Partition 1 (Invalid - Too Low):** < 0 (e.g., Test -5)
- **Partition 2 (Valid):** $0 \leq$ speed ≤ 100 (e.g., Test 45)
- **Partition 3 (Invalid - Too High):** > 100 (e.g., Test 150)

Instead of 10,000 tests, we mathematically prove the function works using only 3 tests!

Black-Box: Boundary Value Analysis

The Reality: Bugs hide at the boundaries! Programmers frequently use < when they meant <=.
The Solution: Instead of picking a random value in the equivalence partition (like 45), test the exact boundaries, plus and minus one increment.

For the speed variable (0 to 100), the absolute most critical test values are: -1, 0, 1, 99, 100, 101.

White-Box: Code Coverage

If you can see the code, you must ensure your tests actually execute every part of it. We measure this using **Coverage Metrics**.

```
    def check_permissions(user_role):
        if user_role == "Commander":
            return True
        else:
            return False
```

- **Statement Coverage:** Does the test suite execute every single line of code at least once?
- **Branch/Decision Coverage:** Do your tests force the program to travel down *both* the True path and the False path of every `if` statement?
- **Path Coverage:** Does the test suite execute every possible combination of paths through the program? (Extremely thorough, but computationally expensive for large systems).

Industry standard typically mandates at least 80% Statement and Branch coverage for production software.

Agenda

1 Recap & Introduction

2 The Philosophy of Testing

3 Testing Levels & The V-Model

4 Test Design Techniques

5 Writing Tests & Mocking

6 Testing Types

7 Test Automation & CI/CD

8 Next Steps

Anatomy of a Unit Test (The AAA Pattern)

Good unit tests are highly structured and easy to read. We follow the **Arrange, Act, Assert (AAA)** pattern.

```python
# The logic (main.py)
def calculate_battery_drain(distance, payload_weight):
    return (distance * 0.5) + (payload_weight * 0.2)

# The test using pytest (test_main.py)
def test_calculate_battery_drain():
    # 1. ARRANGE: Set up the inputs and expected outputs
    distance = 10
    payload = 5
    expected_drain = 6.0

    # 2. ACT: Call the actual function
    actual_drain = calculate_battery_drain(distance, payload)

    # 3. ASSERT: Verify the result matches expectations
    assert actual_drain == expected_drain
```

The Problem: External Dependencies

What happens if the function we want to test talks to a Database or an external Robot API?

```
import requests

def move_physical_robot(x, y):
    # This makes a REAL network request to hardware!
    response = requests.post("http://robot-api:5000/move", json={"x":x, "y":y})
    return response.status_code == 200
```

Why is this fatal for Unit Testing?

- It makes the test **slow** (network latency).
- It makes the test **flaky** (if the robot API is down, your backend test fails, even though your backend code is perfectly fine!).
- **It has physical consequences:** You don't want an automated test moving a real robot 100 times a day in the lab!

The Solution: Mocking & Stubbing

We use **Mocks** (or Test Doubles) to intercept the external call and return a fake, predictable response instantly.

```
from unittest.mock import patch
import my_robot_module

# We use the @patch decorator to intercept the 'requests.post' function
@patch('my_robot_module.requests.post')
def test_move_physical_robot(mock_post):
    # Arrange: Tell the mock to pretend it returned a 200 OK
    mock_post.return_value.status_code = 200
    # Act: Call our function (it will hit the mock, NOT the real internet!)
    result = my_robot_module.move_physical_robot(50, 50)
    # Assert: Verify our code processed the fake data correctly
    assert result == True
    # We can also assert the mock was called with the right data!
    mock_post.assert_called_with("http://robot-api:5000/move",
            json={"x":50, "y":50})
```

Mocking is absolutely essential for testing modern Microservices and Containerised environments!

Agenda

1 Recap & Introduction

2 The Philosophy of Testing

3 Testing Levels & The V-Model

4 Test Design Techniques

5 Writing Tests & Mocking

6 Testing Types

7 Test Automation & CI/CD

8 Next Steps

Testing Types: Functional vs. Non-Functional

Functional Testing

Validates the software against functional requirements. ”Does it do what it is supposed to do?”

- **Smoke Testing:** A shallow, rapid test of crucial functions to see if the build is totally broken before spending time on deep testing.
- **Regression Testing:** Re-running old tests to ensure a new feature didn’t break an old feature. (Automated tests make this painless).

Non-Functional Testing

Evaluates the operational characteristics. ”How well does it do it?”

- **Load/Performance Testing:** Pushing thousands of simulated users to see when the server crashes.
- **Security Testing:** Penetration testing, checking for SQL injections.
- **Usability Testing:** HCI checks (as discussed in Week 6).

Agenda

1 Recap & Introduction

2 The Philosophy of Testing

3 Testing Levels & The V-Model

4 Test Design Techniques

5 Writing Tests & Mocking

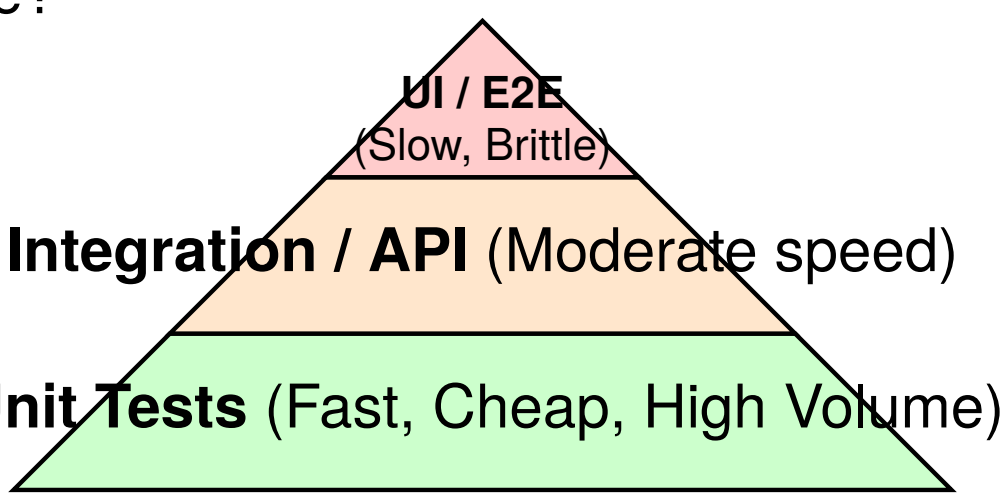
6 Testing Types

7 Test Automation & CI/CD

8 Next Steps

The Test Automation Pyramid

Writing tests takes time. How should we distribute our automation efforts to maximize coverage while minimizing execution time?



- Write **hundreds** of Unit tests (they execute in milliseconds).
- Write **dozens** of Integration tests (they require databases/containers to spin up).
- Write **very few** End-to-End UI tests (using tools like Selenium/Cypress). They are incredibly slow and break whenever a button changes color.

Continuous Integration (CI)

The Problem: Developers write tests locally, but forget to run them before merging code, resulting in a broken `main` branch that affects the whole team.

The Solution: Continuous Integration (CI).

- CI is the practice of automating the integration of code changes from multiple contributors into a single software project.
- Every time code is pushed (e.g., via a Pull Request on GitHub), a remote cloud server automatically pulls the code, builds the Docker containers, and runs the entire test suite.
- If a single test fails, the CI pipeline turns **RED** and the code is blocked from being merged.

Continuous Delivery vs. Continuous Deployment (CD)

If CI ensures the code is safe, **CD** ensures the code gets to the user efficiently.

The complete CI/CD Pipeline:

1. **Code & Push:** Developer pushes to GitHub.
2. **Build (CI):** Server builds the Docker Image.
3. **Test (CI):** Server runs Unit and Integration tests.
4. **Release (Continuous Delivery):** Docker image is pushed to an artifact Registry (like DockerHub or GitHub Container Registry) and waits for human approval to go live.
5. **Deploy (Continuous Deployment):** Completely automated. The production server pulls the new image and swaps it with the old one without human intervention.

Companies like Amazon and Netflix deploy thousands of times a day using this exact architecture!

Agenda

1 Recap & Introduction

2 The Philosophy of Testing

3 Testing Levels & The V-Model

4 Test Design Techniques

5 Writing Tests & Mocking

6 Testing Types

7 Test Automation & CI/CD

8 Next Steps

This Week's Lab: Writing Tests & CI

In the workshop, you will move from theory to practice by implementing automated testing for your Assessment 1 Robot Management System.

- **Task 1: Unit Testing:** Write isolated tests for your backend logic using Boundary Value Analysis.
- **Task 2: API Integration Testing:** Write tests to verify your backend routes return the correct HTTP status codes when handling authorization logic.
- **Task 3: GitHub Actions CI:** Create a YAML pipeline to automatically build your Docker containers and run your tests every time you push to your repository.

Reading & Resources

- **Recommended Reading:** Sommerville, *Software Engineering 9th Edition*, Chapters 8 (Testing) and 25 (Configuration Management).
- **Software Testing, 2nd Edition:** Ron Patton (Sams Publishing).
- **PyTest / Jest Documentation:** Industry standards for writing tests in Python and JavaScript.
- **GitHub Actions Docs:** Review the syntax for creating your own CI/CD `.yml` workflows.

Any Questions?

fdelduchetto@lincoln.ac.uk